



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ 2.ÖDEV RAPORU

Nüfus Simülasyon Oyunu-2

B251210351- Asude Elif Karaağaç

SAKARYA

Mayıs, 2026

Programlama Dillerinin Prensipleri Dersi

Nüfus Simülasyon Oyunu-2

Asude Elif Karaağaç
B251210351 – 2A

Özet

Oyun dinamik bir nüfus değiştirme simülasyonudur. İlk aşamada Java dili Eclipse ortamında oluşturulan yapının Visual Studio Code ortamında C diline farklı benzetimler yönüyle bir nevi dönüştürülmesidir. Java da yerleşik olarak bulunan Kapsülleme ve Bileşim özelliklerini C dilinde struct ve Fonksiyon göstericileriyle manuel olarak benzetilmiştir. Şehirlerin ilçelerden, ilçelerin ise mahallelerden oluştuğu bu "sahiplik" durumu, C'de dinamik bellek yönetimi ve işaretçi dizileri (pointer arrays) kullanılarak yönetilmiştir. Kalıtım benzetimi ise Yerleşim adında bir taban sınıftan da nesne oluşturarak sağlanmıştır. Örnek olarak Şehir sınıfından bir nesne oluşturulur.

Nüfus Simülasyon Oyunu ilk etapta kullanıcıdan tur sayısı ve oyunu başlatan şehir temsili olan 2 basamaklı nüfus sayılarını giriyor. Sonrasında bu sayılar öncesinde oyun kurallarına uyacak formatta artırma işlemlerine tabi tutuluyor. Bu aşamadan sonra ise tur atmalar başlıyor. Turlar sürecinde şehirlerin nüfuslarında artışlar ve 4 basamağı geçmesi durumunda şehrin bölünmesi ve bir kısmının yeni bir şehre taşınması şeklinde ilerliyor. Tüm turlamalar bittiğinde ekranda son turun çıktısı matrisel formatta yazdırılıyor. Sonrasında kullanıcının incelemek istediği şehri seçmesi için matris üzerinden satır ve sütun bilgisi istedik. Buna göre şehir seçiliyor ve seçilen şehrin adı ve ona bağlı ilçeler ve ona bağlı tüm mahallelerde yaşayan kişilerin id-isim soyisim-yaş bilgileri ekranda yazdırılıyor. Bu bilgiler Faker kütüphanesinin kullanımının benzetimi olarak ayrı 10000'lik verilerin metin dosyalarına yazılması ve içerisinden rastgele çekilmesi ile sağlanır. En son ise tuşa basarak program sonlandırılıyor.

© 2017 Sakarya Üniversitesi.

Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içerisinde belirttim. Herhangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler:Şehir,Nüfus,Kişi.

1.Geliştirilen Yazılım

1.1 Başlangıç ve yapıların tanıtımı

İlk olarak VSCode ortamında bin içerisine çalıştırılabilir exe programını, include içerisine başlık dosyalarını, lib içerisine .o dosyalarını ve src içerisine kaynak kodlarını yerleştirecek şekilde klasör düzenimi oluşturdum. Ayrıyeten verileri txt dosyalarının bulunduğu veri_havuzu ve makefile dosyası da proje klasörümde bulunmakta. Yapılarım: Kişi ,Yerleşim, Şehir, İlçe, Mahalle, Main, Oyun ve VeriHavuzu. Main hariç hepsi başlık ve kaynak dosyaları olarak oluşturdum. Main sadece kaynak kod olarak bulunmakta.

Kişi yapısında başlık dosyasında bizden tutmamızın istendiği id, isim, soyisim, yas değişkenlerinin tanımlandığı bir struct yapısı kurdum. String olan isim ve soyisimi char* ile tanımladım. Onun üzerinde ise pointer gizlemesi olarak typedef struct KISI* Kisi satırını ekledim. Struct içerisinde bu yapıya ait olan yasArtir fonksiyonunun göstericisi void (*yasArtir)(struct KISI*) olarak bulunmakta. Sonrasında struct dışına Kurucu fonksiyonu javaya benzetim açısından new_Kisi isminde parametre olarak isim, soyisim yas bilgilerini almakta. yasArtir ve C'nin javadan büyük bir farkı olan Çöp Toplayıcısının eksikliğini gidermek açısından delete_Kisi fonksiyonlar mevcut. Son olarak ise getterlar var. KisiIdGetir, KisiIsimGetir, KisiSoyisimGetir ve KisiYasGetir bunlar ise dolaylı erişim sağlarlar. Kaynak kodunda ise ilk olarak kurucuda javaya benzetimi açısından gövde içerisinde this tanımlaması yaptım. Kisi this = (Kisi)malloc(sizeof(struct KISI)); bellekten this adını kullanarak nesne için yer ayırıyoruz. Ve aynı şekilde isim, soyisim için bellekten yer alıyoruz ve bir de her bir kişi oluşturulduğunda benzersiz bir id veriliyor ve idSayac artırılıyor.

Yerleşim yapısı javadan c ye geçişteki en önemli parçalardan biri. Kalıtım benzetimi buradan yapılmakta bu struct'ın ilk elemanının adresiyle struct'ın kendi adresinin aynı olması ile sağlanmakta. Şehir, İlçe ve Mahalle yapılarının bu yapıyı "base" (taban) olarak kullanır, böylece hepsi ortak bir kimliğe (Yerlesim) sahiptir. Başlık dosyası içerisinde de aynı şekilde typedef satırı ve struct yapısı bulunmakta. Struct içinde ad (char*) değişkeni ve nufusGetir, yokEt gibi sanal fonksiyon göstericileri mevcut. YerlesimOlustur ve YerlesimYokEt fonksiyonlarının imzaları var. Kaynak dosyasında private benzetimi olarak başlık dosyasında imzası bulunmayan static VarsayılanNufusGetir fonksiyonu var varsayılan değer olarak 0 döndürür çünkü alt sınıflar bu fonksiyonu ezecekler. Burada ne kurucu ve yıkıcı içerisinde yerel this tanımlaması yaptım. Kurucuda yerleşim yeri için bellekten alan açılıyor. İsim uzunluğu kadar yeni yer açılır ve strcpy ile isim buraya kopyalanır. Bu şekild enesne kendi verisine sahip olur. Sanal metodların

bağlanması da burada yapılmakta. Nesneye varsayılan davranışlar yüklenir bu Çok Biçimlilik ve Geç Bağlama mekanizmalarının taklididir. Yıkıcı ise önce adını sonra kendisini siler.

Şehir yapısında başlık dosyasında Yerlesim base; ile kalıtım benzetimi yapılmıştır. Typedef ile pointer gizledikten sonra struct içerisinde İlçe* ilçeler; ile javada ArrayList olarak oluşturduğumu burada C dilinde realloc ile yönetilen dinamik dizi (dynamic array) yapısı.olarak oluşturdum. ilçeSayisi, mevcutSayi (yaşayan kişilerin toplamı) ve kapasite değişkenleri tanımlı. Sonrasında fonksiyon göstericileri olarak ilçeEkle, getToplamNufus şehirBol, delete_Sehir mevcut. En son struct dışında new_Sehir, ŞehirİlçeEkle, delete_Sehir fonksiyonlar ve ŞehirAdAyarla, ŞehirİlçeleriAyarla, ŞehirMevcutSayiAyarla, ŞehirMevcutSayiGetir getter ve setter imzası bulunuyor. Kaynak dosyasında ise private benzetimi olara kstatic ile tanımladığım ŞehirToplamNufusHesapla ve şehirleriBol bulunmakta. Yine kurucu içerisinde this tanımlaması, malloc ile bellekten yer ayırma ve fonksiyon göstericilerin bağlanması var. Polimorfizm benzetimi olarak yıkıcı da Yerlesim yıkıcısını şehre bağlıyoruz this->base->yokEt = (void (*)(Yerlesim))&delete_Sehir; en son this i return ediyoruz. Başlıktaki fonksiyonların gövdeleri var. Sadece şehirleriBol de ilçe sayısını 2 ye bölüp bir orta nokta üzerinden ne kadar ilçenin yeni şehre taşınacağı ayarlanıyor. Yıkıcıda önce ilçe dizisinin kendisini, kalıtım alınan base yapısını ve en son şehrin kendisini free() ile temizliyoruz.

İlçe yapısı da şehir yapısına benzer şekilde Başlık dosyasında struct içerisinde Yerlesim base; kullanıyor. Fonksiyon göstericileri var. Struct dışında fonksiyon imzaları ve getter imzaları var. Kaynak kodunda new_Ilçe fonksiyonu, bir ilçenin hem kendi verilerini hem de altındaki mahalleleri yönetecek mekanizmayı kurar. İlçe nesnesi bellekte yer alırken, aynı zamanda altındaki Mahalle nesnelerinin adreslerini tutacağı başka bir bellek alanını da rezerve eder. Bu, Java'daki private List<Mahalle> mahalleler yapısının C'deki karşılığıdır. this->base = YerlesimOlustur(ad); satırı ile İlçe nesnesi, Yerlesim sınıfından türetilmiş bir yapıya bürünür. Bu, C'de Kalıtım için Bileşim yöntemidir. İlçe, bir yerleşim yeridir ve yerleşimin tüm özelliklerini miras alır. this->mahalleler = (Mahalle*)malloc(this->kapasite * sizeof(Mahalle)); satırı, ilçenin içindeki mahalleleri tutacak olan dinamik diziyi oluşturur. this->base->nufusGetir = &I_NufusHesapla; burda yapılan override benzetimidir.

Mahalle yapısı da aynı şekilde oluşturdum. Yaşayan kişileri Kisi* referans listesi olarak tuttum. Kaynak kodundaki MahalleKisiEkle fonksiyonunda kapasite kontrolü yapıyor mevcut ayrılan yerin yetip yetmediği üzerine ona göre 2 katına çıkarıyor ArrayList yapısında olduğu gibi. realloc: Mevcut verileri koruyarak bellekteki alanı büyütür. Eğer yan taraftaki bellek blokları doluyorsa, veriyi otomatik olarak daha geniş bir boş alana taşır.

Oyun yapısında başlık dosyasında yine benzer yapıda struct, fonksiyon göstericileri ve fonksiyon imzaları bulunmakta. Ek olarak rastgele isim, soyisim, şehir adı gibi seçimlerin fonksiyonları buradadır. Oyuna ait kurallar buradadır. Girilen sayıların rakamlarının ne anlama geldiği, mahalle sayısının ilçe sayısına eşit bölünmesi, nüfusun mahallelere eşit bölünmesi ve nüfus 4 basamaklı olduğunda 2 ye bölünmesini uygulayan fonksiyonlar. Önce oyun için bellekten yer ayrılıyor sonrasında OyunSehirolustur içerisinde for döngüleriyle iç içe ilçe, mahalle, yaşayan kişiler oluşturuluyor. Turla fonksiyonu da burada. Tur içerisinde nüfus bölünmeleri ve her turda kişilerin yaşlarının 1 artırılması işlemleri yapıyor. Bölümlerde bir geçici liste oluşturuyoruz sonrasında bu listeyi asıl olana ekliyoruz. Delete_Oyun da ise delete_Sehir yıkıcısını Yerlesim ile çağırıyoruz. Sonra şehirler listesini en son oyunun kendisini bellekten free ile temizliyoruz.

VeriHavuzu rastgele bilgilerin metin dosyalarından çekilmesini sağlıyor. Ve oyun içinde kullandığımız rastgele üret fonksiyonları burada char* olarak tanımlıdır. 5 ayrı metin dosyasıdır. 10000 x 50 boyutlu global karakter dizilerine RAM'e yüklenir. Böylece her rastgele üretim işleminde disk erişimi yerine doğrudan bellekten rand() % 10000 indeksiyle anlık veri çekilir.

Main sınıfında ise oyunu başlatma fonksiyonu ve kullanıcı etkileşimi sağlayan ekran işlemleri bulunmakta. Girdilerin alınması, oyunun başlatılması, turlara hazırlık ve tur sonuçlarının ekrana yazdırılması işlemlerinin yanında ŞehirleriListele ve SecimEkranı fonksiyonları bulunmakta. Şehirleri listeler ekrana 5'erli olarak köşeli parantezler içerisinde nüfus sayısını yazdırırken secimEkranı kısmında ise şehir seçiliyor ve seçilen o şehirde yaşayan tüm kişiler listeleniyor.

1.2 Sistemin işleyişi

Yukarda bahsettiğimiz sınıfların bir arada çalışmasıyla birlikte program çalışmaya başladığı anda ilk başta tur bilgisinin girilmesini istiyor. Sonrasında ise 2 basamaklı sayılar giriliyor, bunların her biri birer şehir ve değerleri ise o şehrin nüfusu anlamına gelmekte. Onlar basamağı ilçe sayısı birler basamağı ise mahalle sayısı anlamına gelmektedir.

Turlar başlamadan önce bir hazırlık evresi var. Burada istenen nüfusun mahallelere eşit oranda paylaştırılmasını sağlamak için arttırma işlemi uygulamak ve dikkat ettiğimiz önemli bir nokta ise bu artışın içe ve mahalle üzerinde herhangi bir değişiklik

olmaması yönünde. İlçelerde mahalleleri eşit paylaşıcağından her yerde eşit paylaşım söz konusudur. İlçe sayısı 0 olamazken mahalle olursa direkt ilçe sayısına eşitledim.

Hazırlık aşamasında ilçeler mahalleleri eşit bölüşeceği için mahallenin ilçeye bölümü tam sonuç vermiyorsa bölünebilen en yakın sayıya arttırılacaktır. Burada birler basamağında 9'dan fazla bir değer gelirse ilçeye dokunmadan mahalle sayısının tekrardan 1'den itibaren arttırılarak bölünebilen sayıya ulaşılması sağlanır.

Tüm bu ayarlamalardan sonra Faker Kütüphanesinin benzetimi olarak oluşturduğum Veri_Havuzu klasörümdeki 10000'er veri içeren metin dosyalarını kullanarak zincirleme şekilde önce şehir sonrasında girilen değere göre ilçeler, mahalleler ve eşit oranlı paylaşılmış kişilerin oluşturuluyor.

Hazırlık bittiğinde durumlar köşeli parantezler içinde yazdırılır ve hemen ardından tur işlemleri başlar.

Turlardaki işleyiş her turda nüfusun belli bir artış oranıyla artacağı ve eğer ki 4 basamağı geçtiği takdirde nüfusun ilçe ve mahalleyle birlikte belli bir oranda paylaşarak yeni bir şehre taşınması şeklindedir.

Artış oranı her zaman her turla birlikte değişmekte olup, onlar ve birler basamaklarının toplamlarıyla elde edilmektedir. Eğer ki toplam 0 olursa o zamanda artış oranı 1 gibi düşünülür. Nüfus bu artış oranıyla çarpılmakta ve eğer ki 4 basamaklı bir değere ulaşırsa çift olması halinde direkt tam yarısı ve tek olması durumunda ise yarısının 1 eksiği oranıyla **(9 ise 5 eskisinde kalacak 4'ü yeni şehre taşınacak)** bölüşerek az olan kısmın yeni şehre taşınmasını içerir. Yeni oluşan şehirleri de listenin sonundan ekler.

Bu kısımda ödev yönergesinden ilk başta bende bıraktığı izlenim ekranda herhangi 4 basamaklı bir sayının yazılmayacağı şeklindeydi. Bundan sebep her turun içerisinde while döngüsü ile 4 basamaklı bir değer kalmayana kadar bölme işlemi uygulamış ve turun o şekilde tamamlanmasını sağlamıştım ama bu şekilde şehir sayısı çok fazla artmakta ve sürekli aynı değerler gözükmekteydi. Sonradan yapılan duyuru ile tur başı 1 bölünme gerçekleştirileceği şeklinde düzenledim.

Javadan C'ye dönüşümde en çok zorlandığım alan biri de, ekranda milyonlu nüfus sayıları gibi anlamsız değerlerin görünmesi ve programın beklenmedik anlarda çökmesi idi. Bu durumun iki temel sebebi olduğunu fark ettim ve şu şekilde çözüm sağladım:

Yıkıcı Fonksiyonlarda Sonsuz Döngü: Java'daki çöp toplayıcı yerine C'de manuel bir yıkıcı mekanizması kurarken, taban sınıf (Yerlesim) ile alt sınıfların (Şehir, İlçe, Mahalle) yokEt göstericilerini birbirine bağlamıştım. Ancak alt sınıfın içinden this->base->yokEt(this->base) çağırısı yaptığımda, gösterici tekrar alt sınıfın yıkıcısına döndüğü için sonsuz bir döngüye giriliyor ve bellek bozuluyordu. Bu sorunu, alt sınıfların içinde taban sınıfı temizlerken fonksiyon göstericisini değil, doğrudan YerlesimYokEt fonksiyonunu çağırarak çözdüm.

Fonksiyon Göstericileri ve Parametre Uyuşmazlığı: Çok biçimlilik simülasyonu yaparken, fonksiyon göstericilerinin beklediği parametre tipleri ile gönderilenler arasında (örneğin Yerlesim yerine Mahalle pointer'ı) uyumsuzluklar oluşuyordu. Bu durum, belleğin yanlış adreslerine erişilmesine ve "milyonlu" hatalı nüfus verilerine yol açıyordu. Bu problemi, fonksiyon atamaları sırasında açık tür dönüşümü kullanarak ve pointer listelerini (özellikle Kisi gibi yapılar) daha dikkatli yöneterek aştım.

2.Çıktı

Tur sayisini girin: 1
Oyun formatina uyacak sekilde bosluklu giris yapiniz(ör:18 25 79):
18 25 79 37 62

Turlara hazırlık sonu durumu:
[24]-[30]-[77]-[45]-[66]

-----1.Tur-----
[144]-[90]-[616]-[405]-[792]
[462]

----- OYUN SONU DURUM -----

[144]-[90]-[616]-[405]-[792]
[462]

===== SEHIR DETAYLI SORGULAMA EKRANI =====

SATIR numarasini gir(0'dan baslayarak): 0
SUTUN numarasini gir(0'dan baslayarak): 1

Şehir: Van - Nufus: 90
İlçe: Michigan - Nufus: 45
Mahalle: Zengel Spurs - Nufus: 15
Kisiler:

3.Sonuç

Bu proje ile Java'daki OOP mimarisinin C dilinde düşük seviyeli bellek yönetimi ve fonksiyon göstericileriyle nasıl simüle edilebileceği deneyimlenmiş, dinamik veri yapılarının yönetimi konusunda yetkinlik kazandım.

Düzeltmeye uğraştığım hatalar başlangıçta moral bozucu olsa da, C dilinde bellek yerleşimi ve nesne hiyerarşisinin arka planında neler döndüğünü anlamamı sağlayan en öğretici kısımlar oldu.